

1.

La capacidad de emitir múltiples instrucciones en un único ciclo de reloj. Para lograrlo, estos procesadores disponen de múltiples unidades funcionales independientes que pueden operar en paralelo.

Su CPI (Ciclos Por Instrucción) ideal es menor que uno. Esto se justifica porque, al emitir y ejecutar más de una instrucción por ciclo, el tiempo promedio dedicado a cada instrucción se reduce por debajo de la unidad (por ejemplo, en un procesador de emisión doble, el CPI ideal sería 0.5).

2.

Existen tres categorías fundamentales:

Superescalares de planificación estática: Emiten un número variable de instrucciones por ciclo y dependen del compilador para el ordenamiento y la prevención de riesgos.

Procesadores VLIW (Very Long Instruction Word): Emiten un número fijo de instrucciones empaquetadas en una sola palabra larga, con el paralelismo indicado explícitamente por el compilador.

Superescalares de planificación dinámica: Utilizan hardware avanzado (como el algoritmo de Tomasulo) para emitir y ejecutar instrucciones fuera de orden, gestionando los riesgos en tiempo de ejecución.

3.

La arquitectura se basa en múltiples unidades funcionales independientes. Las instrucciones son muy largas (80-120 bits o más) porque contienen campos específicos para cada unidad funcional. Entonces el compilador analiza el código, identifica las instrucciones que pueden ejecutarse en paralelo y las empaqueta en una única "instrucción muy larga" o paquete. El hardware es más simple que en los superescalares dinámicos porque no necesita detectar riesgos; confía plenamente en que el compilador ha organizado las instrucciones de forma segura.

4.

- **Aumento del tamaño del código:** Debido al uso intensivo de técnicas como el desenrollado de bucles y a que los huecos no utilizados en la instrucción larga se rellenan con operaciones nulas (NOP), lo que desperdicia bits.
- **Rigidez técnica (Lockstep):** Históricamente, si una unidad funcional se detenía (por ejemplo, por un fallo de caché), todo el procesador debía detenerse para mantener la sincronización.

- **Compatibilidad binaria:** El código está ligado a la estructura interna del procesador (número de unidades y sus latencias). Si se cambia la arquitectura, el código antiguo no suele funcionar o no aprovecha el nuevo hardware sin ser recompilado.

5.

- **Desenrollado de bucles (Loop Unrolling):** Aumenta el cuerpo del bucle para exponer más paralelismo a nivel de instrucción (ILP) y rellenar los huecos de la palabra larga.
- **Planificación global y de trazas (Trace Scheduling):** Algoritmos complejos que permiten mover instrucciones a través de ramas condicionales para maximizar la ocupación de las unidades funcionales.
- **Software Pipelining:** Una técnica que permite obtener los beneficios del desenrollado sin aumentar tanto el tamaño del código.
- **Compresión de instrucciones:** Para mitigar el crecimiento del tamaño del código en memoria.

6.

- **Arquitectura y Funcionamiento:** Utiliza una extensión del **algoritmo de Tomasulo** que incluye una **estación de reserva** para cada unidad funcional y un **Buffer de Reordenamiento (ROB)**. Permite emitir instrucciones en orden, ejecutarlas fuera de orden (tan pronto como sus operandos estén listos) y confirmarlas (commit) de nuevo en orden para mantener la semántica del programa.
- **Complejidad:** Radica en la lógica de emisión. El hardware debe analizar en paralelo todas las dependencias entre las instrucciones de un mismo paquete de emisión en un solo ciclo de reloj. La complejidad crece con el cuadrado del número de instrucciones emitidas simultáneamente.

7.

Los superescalares dinámicos son más adecuados para código irregular porque pueden tomar decisiones en tiempo de ejecución. Mientras que un VLIW depende de una planificación estática hecha por el compilador, el hardware dinámico puede saltarse instrucciones bloqueadas y ejecutar otras independientes, aprovechando mejor los recursos cuando el flujo del programa es poco predecible.

8.

Son antónimos en concepto pues el superescalar dinámico depende exclusivamente del hardware mientras el VLIW depende exclusivamente del compilador. El primero el compilador puede importar para agilizar el proceso, pero el encargado de organizar la ejecución es el hardware. Por otro lado, el VLIW organiza en base a tags y ejecuta múltiples instrucciones en el mismo ciclo a través de una organización mediante el compilador (ni tiene detección de riesgos el hardware).

